From Counsel Guidance to Executable Review Rules

Ingrid Kao

Independent practitioner note | November 11, 2026

Companion files · GitHub

At a glance

Built: A versioned NDA policy engine evaluating source-linked annotations.

Observed (synthetic): 18 scenarios produced 36 evaluations, exposing four outcome changes and six reason changes.

Meaning: Policy changes remain separate from contract facts, with review freshness checked independently.

At Lawsnote, I turned an enterprise client's NDA review standards into rules that engineers built into an AI contract-review product. A legal review instruction can sound precise until someone tries to implement it. Require three years of confidentiality. Count from which event? Does the instruction cover source code? What should happen when two clauses describe different survival periods? A developer who supplies those answers has changed the review policy, even if the resulting software runs without errors.

I propose treating each review rule as a maintained legal decision: it needs a defined scope, evidence requirements, an outcome, and a person authorized to change it. The companion prototype demonstrates this approach through two fictional policy versions and an inspectable set of contract annotations. The most useful result is a change report that explains which decisions move, which remain stable, and which cannot be resolved from the supplied facts.

Resolve the policy question before implementing the condition

Consider a fictional sponsor reviewing NDAs for Project Atlas. Its initial instruction requires at least 36 months of confidentiality from disclosure. A revised instruction requires 60 months for source-code disclosures while retaining 36 months for ordinary information. Separate rules restrict use to evaluating Project Atlas and require escalation of unrestricted residual-knowledge rights. These are invented review positions, not legal requirements or claims about market practice.

The duration change leaves a question that the numbers cannot answer: how should a reviewer handle an agreement covering both ordinary information and source code? The sponsor could apply the longer requirement throughout, split the obligations by category, or require a specific decision. The demonstration takes the third approach for mixed or unknown classifications under the revised rule. That choice is visible in the configuration rather than hidden inside a default branch.

Duration also needs a starting event. Three years from the effective date may expire earlier than three years from a later disclosure. The prototype therefore evaluates an annotated start event as well as the month count. An uncertain or conflicting material fact goes to review. A number reaching the threshold cannot resolve an unanswered interpretation question.

Before implementation, counsel should approve the scope and exception treatment in language that another reviewer can apply. Engineering can then implement the agreed comparison. A disagreement about the appropriate confidentiality period returns to the policy owner; a disagreement about whether the code correctly compares 35 with 36 is a software question. Keeping those decisions distinct makes the next change easier to review.

Keep the source fact separate from the review position

A contract states an obligation. A review policy states what the sponsor wants to do about it. Those records should remain separate. Otherwise, changing a preferred position can silently alter the apparent contents of the agreement.

The companion sources contain short fictional clauses with deliberately authored annotations. Each annotation supplies a value and a reference to its supporting clause and source version. Annotations must identify unresolved or conflicting facts; the engine does not discover those conflicts by interpreting the clauses. The program evaluates the supplied annotations. It does not read an arbitrary NDA and establish its meaning, infer that source code will be disclosed, or decide whether a quoted clause is enforceable.

The same separation matters when a model produces the annotations. A model-generated value is a proposed interpretation until the chosen review process accepts it. Exact quotation checks can establish that words occur in a cited clause; they cannot establish that those words support the normalized value. An accepted extraction layer and the policy engine need separate evaluation.

The rule record should make the implementing team's remaining choices visible:

Rule component	Question the record should answer
Scope and required facts	Which agreements and information categories are covered, and what must be known?
Condition and exceptions	What comparison applies, including the start event and treatment of mixed information?
Outcome and reason	What happens next, and what explanation does the receiving reviewer get?
Authority and owner	Who may approve the rule, decide a departure, and maintain its examples?
Version and application date	Which instruction applied to this review, and when should a replacement be used?
Source and examples	What approved instruction and boundary cases explain the intended behavior?

The prototype implements the maintenance record’s versioned settings, source-linked annotations, decisions, reasons, and comparison reports.

Give uncertainty a useful destination

A binary pass-or-fail result loses information that the legal team needs. A known departure from a review position differs from a missing fact. An agreement outside the rule's scope differs from one whose terms match it.

The demonstration uses four outcomes:

Outcome	Meaning in the demonstration	Next decision
POLICY_MATCH	The supplied facts satisfy the configured instructions evaluated	Continue the applicable review process; this is not approval of the agreement
ESCALATE	A supported annotation establishes a configured policy departure	Obtain the designated decision on that departure
NEEDS_REVIEW	Material facts, source linkage, or interpretation are unresolved	Resolve the identified question before relying on that rule
OUT_OF_SCOPE	The supplied agreement type is outside this NDA rulebook	Route under the appropriate review process

The aggregation rule also needs a decision. Here, a confirmed policy departure produces ESCALATE even if another rule needs more information. The report retains the unresolved rule instead of hiding it behind the aggregate result. With no confirmed policy departure, an unresolved required fact produces NEEDS_REVIEW. Only the evaluated, in-scope instructions can produce POLICY_MATCH.

This design prevents an absent classification from becoming ordinary information by accident. It also preserves useful work: a confirmed overbroad purpose can reach counsel while the reviewer investigates an uncertain duration. A production team would need a queue, accepted receiving owners, and handling instructions for those outcomes. The prototype generates records for inspection rather than sending notifications or resolving the matters.

Compare a policy change against the same cases

The revised source-code threshold should change the intended cases without changing ordinary-information cases merely because they share the duration field. Applying both policy versions to the same pinned annotations makes that expectation testable.

A source-code case with 36 months from disclosure can match the original duration instruction and require escalation under the revised one. An ordinary-information case with the same duration should retain its outcome. A 60-month source-code case can retain its outcome while its explanation changes to identify the revised threshold. Outcome changes and reason changes therefore deserve separate columns.

The change report is a comparison of hypothetical review results under two supplied versions. It does not decide whether a new policy applies retroactively, revoke a negotiated exception, or change an executed agreement. A policy owner must decide which new reviews, pending negotiations, renewals, or existing matters fall within a change. That application decision belongs with the release record.

There is also a difference between an unchanged result and a reusable earlier review. The demonstration's freshness check asks whether a prior record matches the source and policy selected for the present review context. A mismatch requests another check. It does not declare the earlier decision invalid for the context in which it was made.

NIST's configuration-management guidance describes baselines, controlled changes, and impact analysis for information-system security.[1] I adapt the maintenance principle here: preserve the original

configuration and examine a proposed change before substituting it. This is a design analogy, not a claim that the prototype implements that security framework.

Test the boundary and preserve the result

The companion implementation uses Python's standard library and JSON policy files. Its deliberately authored scenarios exercise values around the thresholds, ordinary and source-code categories, incomplete or conflicting facts, uncertain start events, source-linkage failures, and agreement scope. Expected results are recorded separately from the evaluations that the runner produces.

The run evaluated 18 synthetic scenarios under both policies, producing 36 policy evaluations and 108 individual rule rows. All 36 aggregate evaluations matched the authored expectations. Under the original policy, seven cases returned POLICY_MATCH, four ESCALATE, six NEEDS_REVIEW, and one OUT_OF_SCOPE. Under the revised policy, the counts were three, six, eight, and one respectively.

Four cases changed aggregate outcome: the 36- and 59-month source-code cases became policy departures, while mixed and unknown information categories at 36 months required review. Six cases changed rule reasons. That count includes two cases with unchanged outcomes: source code at 35 months remained a departure, and source code at 60 months still matched, but their explanations identified the revised threshold.

Four separate freshness checks produced one CURRENT result and three RECHECK_REQUIRED results for changed source, changed policy, or both. A repeatability check ran twice in fresh directories and compared all seven generated output files byte for byte in the same runtime. These are fixture-level software observations, with review freshness reported separately from the policy evaluations.

These observations establish the configured decision logic on the supplied fixtures, with extraction and substantive review evaluated separately.

Policy-testing documentation from Open Policy Agent illustrates the value of executable examples when rules change.[2] The companion uses Python rather than OPA or Rego. The transferable practice is to retain tests that explain a rule, including cases that should remain unchanged after a revision. Altering an expected answer to match new code still requires a reason and an authorized policy decision where substance changes.

Release a rule with its maintenance obligations

Before release, the rule owner should confirm the approved instruction, affected review contexts, unresolved exceptions, regression results, and a fallback path. The receiving reviewer needs examples of both a policy departure and an unresolved fact. A short explanation should show what evidence resolves each case and who may accept a departure.

Feedback should identify the rule and source version, the disputed outcome, and the proposed correction. The owner can then distinguish a bad annotation, a coding defect, an incomplete instruction, and a deliberate change in risk tolerance. Each requires a different repair. The feedback record should retain the original result so that a later reviewer can reconstruct the change.

For a rulebook used within an AI workflow, NIST's AI RMF 1.0 supports planned ongoing review and documented responsibilities.[3] A practical release record can apply those principles by naming the

owner, review trigger, approval reference, and examples that must be rerun. A new information category, repeated interpretation disagreement, or changed business use should trigger a policy discussion before expanding the existing rule.

The maintained rulebook gives legal judgment a form that a product team can implement and a reviewer can challenge. The next release should preserve why the rule exists, which facts it depends on, and the evidence that its revision changed the intended cases.

Method and references

This independent practitioner study uses original fictional NDA clauses, authored annotations, and illustrative sponsor policies. Its 18 scenarios are a separate deterministic dataset. The engine evaluates annotations rather than interpreting arbitrary agreements; it makes no model calls and does not establish extraction accuracy, legal correctness, enterprise approval authority, reviewer-time savings, or production adoption. Policy comparisons, review-freshness checks, and operational release decisions remain separate.

[1] NIST. Guide for Security-Focused Configuration Management of Information Systems. SP 800-128, August 2011, updated October 2019. Sections on configuration change control and impact analysis. https://doi.org/10.6028/NIST.SP.800-128

[2] Open Policy Agent. Policy Testing. Official documentation, accessed November 11, 2026. https://www.openpolicyagent.org/docs/policy-testing

[3] NIST. Artificial Intelligence Risk Management Framework. AI RMF 1.0, January 2023. GOVERN 1.5 and 2.1. https://doi.org/10.6028/NIST.AI.100-1